

Name: AKSHARAA BALASUBRMANIAN

Register Number: 192311278

Assigned Question: Q1. N-Queens Problem Design a pseudocode using Backtracking to solve the N-Queens problem for an $N \times N$ chessboard. Clearly identify inputs (board size) and expected output (valid queen placements). Develop a logically correct algorithm that ensures no two queens attack each other. Use appropriate control structures such as loops and conditional checks for row, column, and diagonal safety. Ensure the pseudocode is well-structured and easy to follow. Handle all possible configurations and optimize

1. Problem Statement

The **N-Queens Problem** is the problem of placing **N queens on an $N \times N$ chessboard** such that no two queens attack each other.

Since a queen can attack another queen in the same:

- Row
- Column
- Main diagonal
- Secondary diagonal

the algorithm must ensure that every newly placed queen is safe from all previously placed queens.

The **Backtracking technique** is used to systematically try possible positions and **prune invalid placements immediately**, rather than exploring configurations that cannot lead to a valid solution.

2. Objective

To design a backtracking algorithm that:

1. Accepts the board size **N** as input.
2. Places exactly **N queens** on the board.
3. Ensures that no two queens share the same row.
4. Ensures that no two queens share the same column.
5. Ensures that no two queens lie on the same diagonal.
6. Finds and displays **all possible valid queen placements**.
7. Uses pruning to avoid unnecessary searches.

3. Input

Input: Integer N

Where:

- N = size of the chessboard.
- The board contains $N \times N$ cells.
- Exactly N queens must be placed.

Example:

Input:

N = 4

4. Expected Output

The algorithm should display all valid arrangements of N queens.

For N = 4, there are **2 valid solutions**:

Solution 1

. Q . .

. . . Q

Q . . .

. . Q .

Solution 2

. . Q .

Q . . .

. . . Q

. Q . .

Here:

- Q $\rightarrow$ Queen
- . $\rightarrow$ Empty position

5. Backtracking Approach

The algorithm places queens **one row at a time**.

For each row:

1. Try every column.
2. Check whether placing a queen at that position is safe.
3. If safe, place the queen.
4. Move to the next row.
5. If no valid position exists in the next row:
 - Remove the previously placed queen.
 - Try another column.
6. Continue until all queens are placed.

This removal and retry operation is called **backtracking**.

6. Safety Conditions

Before placing a queen at position (row, col), three conditions must be checked.

Condition 1: Same Column

No previous queen should be present in the same column.

`board[i][col] == 1`

Condition 2: Upper-Left Diagonal

Check positions diagonally toward the upper-left:

`i-- , j--`

Condition 3: Upper-Right Diagonal

Check positions diagonally toward the upper-right:

`i-- , j++`

If none of these positions contains a queen, the position is safe.

7. Pseudocode

Algorithm: N-Queens Using Backtracking

ALGORITHM N_QUEENS(N)

INPUT:

$N \leftarrow$ Size of the chessboard

OUTPUT:

All valid arrangements of N queens

BEGIN

IF $N \leq 0$ THEN

 PRINT "Invalid board size"

 RETURN

END IF

CREATE board[N][N]

INITIALIZE every cell of board to 0

solutionCount $\leftarrow$ 0

CALL PLACE_QUEEN(0)

IF solutionCount = 0 THEN

 PRINT "No valid solution exists"

END IF

END

8. Queen Placement Procedure

PROCEDURE PLACE_QUEEN(row)

BEGIN

IF row = N THEN

 solutionCount $\leftarrow$ solutionCount + 1

 PRINT "Solution ", solutionCount

 PRINT board

```

    RETURN
END IF
FOR col  $\leftarrow$  0 TO N - 1 DO
    IF IS_SAFE(row, col) = TRUE THEN
        board[row][col]  $\leftarrow$  1
        CALL PLACE_QUEEN(row + 1)
        board[row][col]  $\leftarrow$  0
    END IF
END FOR
END

```

9. Safety Checking Procedure

```

FUNCTION IS_SAFE(row, col)

```

```

BEGIN

```

```

    // Check the same column

```

```

    FOR i  $\leftarrow$  0 TO row - 1 DO

```

```

        IF board[i][col] = 1 THEN

```

```

            RETURN FALSE

```

```

        END IF

```

```

    END FOR

```

```

    // Check upper-left diagonal

```

```

    i  $\leftarrow$  row - 1

```

```

    j  $\leftarrow$  col - 1

```

```

WHILE i >= 0 AND j >= 0 DO
    IF board[i][j] = 1 THEN
        RETURN FALSE
    END IF
    i ← i - 1
    j ← j - 1
END WHILE

// Check upper-right diagonal
i ← row - 1
j ← col + 1
WHILE i >= 0 AND j < N DO
    IF board[i][j] = 1 THEN
        RETURN FALSE
    END IF
    i ← i - 1
    j ← j + 1
END WHILE
RETURN TRUE
END

```

10. Complete Combined Pseudocode

For submission, you can use this as the **main pseudocode**:

ALGORITHM SOLVE_N_QUEENS(N)

INPUT:

$N \leftarrow$ Board size

OUTPUT:

All valid placements of N queens

BEGIN

IF $N \leq 0$ THEN

 PRINT "Invalid board size"

 RETURN

END IF

CREATE board[N][N]

INITIALIZE all board cells to 0

solutionCount $\leftarrow$ 0

FUNCTION IS_SAFE(row, col)

 // Check column

 FOR $i \leftarrow 0$ TO row - 1 DO

 IF board[i][col] = 1 THEN

 RETURN FALSE

```
    END IF  
  END FOR
```

```
// Check upper-left diagonal
```

```
i ← row - 1
```

```
j ← col - 1
```

```
WHILE i ≥ 0 AND j ≥ 0 DO
```

```
  IF board[i][j] = 1 THEN
```

```
    RETURN FALSE
```

```
  END IF
```

```
  i ← i - 1
```

```
  j ← j - 1
```

```
END WHILE
```

```
// Check upper-right diagonal
```

```
i ← row - 1
```

```
j ← col + 1
```

```
WHILE i ≥ 0 AND j < N DO
```

```
  IF board[i][j] = 1 THEN
```

```
    RETURN FALSE
```

```
  END IF
```


$i \leftarrow i - 1$

$j \leftarrow j + 1$

END WHILE

RETURN TRUE

END FUNCTION

PROCEDURE BACKTRACK(row)

IF row = N THEN

 solutionCount $\leftarrow$ solutionCount + 1

 PRINT "Valid Solution ", solutionCount

 PRINT board

 RETURN

END IF

FOR col $\leftarrow$ 0 TO N - 1 DO

 IF IS_SAFE(row, col) = TRUE THEN

```

// Place queen
board[row][col] ← 1

// Recursively place queen in next row
CALL BACKTRACK(row + 1)

// Backtrack and remove queen
board[row][col] ← 0

END IF

END FOR

END PROCEDURE

CALL BACKTRACK(0)

IF solutionCount = 0 THEN
    PRINT "No valid solution exists for N = ", N
ELSE
    PRINT "Total Solutions = ", solutionCount
END IF

END

```

11. Working Example: N = 4

Initially:

....

....

....

....

The algorithm starts from **row 0**.

It tries:

Row 0 → Column 0

Queen is placed:

Q...

....

....

....

Now the algorithm moves to row 1.

Some positions are rejected because they are attacked by the queen.

The algorithm continues trying safe positions.

When it reaches a situation where **no safe position is available**, it performs:

Remove previous queen

↓

Go back to previous row

↓

Try another column

This is the **backtracking process**.

Eventually, one valid configuration is obtained:

. Q ..

... Q

Q ...

.. Q .

The algorithm continues searching to find the remaining configurations.

12. State-Space Representation

The search can be visualized as:

[illegible]

Whenever an unsafe position is encountered:

Unsafe Position

↓

Prune



Try next column

This **pruning** reduces the number of configurations explored.

13. Why Backtracking is Used

A brute-force approach would generate a large number of possible arrangements and check them only after placing all queens.

Backtracking is more efficient because it checks constraints **during the construction of the solution**.

For example:

Place Queen



Check Safety



Safe? — No —→ Reject immediately



Yes



Move to next row

Therefore, invalid partial solutions are discarded as early as possible.

14. Correctness of the Algorithm

The algorithm is correct because:

Row Constraint

Only one queen is placed during each recursive call, and the algorithm moves to the next row after placing a queen.

Therefore, **no two queens occupy the same row**.

Column Constraint

IS_SAFE() checks the entire previous portion of the same column.

Therefore, **no two queens occupy the same column**.

Diagonal Constraint

The algorithm checks both:

- Upper-left diagonal
- Upper-right diagonal

Therefore, **no two queens occupy the same diagonal.**

Complete Search

Every column in every row is considered unless it is immediately rejected as unsafe.

Therefore, all possible valid configurations can be found.

15. Complexity Analysis

Time Complexity

The worst-case time complexity is approximately:

$O(N!)$

because the algorithm explores possible arrangements while pruning invalid states.

Space Complexity

The board requires:

$O(N^2)$

space.

The recursive call stack requires:

$O(N)$

additional space.

Therefore, total auxiliary space is approximately:

$O(N^2)$ due to the board representation.

16. Advantages

- Uses systematic **backtracking**.
- Eliminates invalid positions early.
- Finds all possible solutions.
- Easy to understand and implement.

- Demonstrates recursion and constraint satisfaction.
- More efficient than checking every complete board arrangement.

17. Limitations

- Execution time increases rapidly as N becomes large.
- The number of possible configurations grows very quickly.
- Basic safety checking scans rows/diagonals repeatedly.

For larger values of N, the algorithm can be optimized using additional arrays/sets for:

Column

Main Diagonal

Secondary Diagonal

This can reduce the safety-check operation from **O(N)** to **O(1)** per placement.

18. Optimized Backtracking Pseudocode

If your faculty specifically asks for **optimization by pruning**, this version is even better:

ALGORITHM OPTIMIZED_N_QUEENS(N)

INPUT:

N ← Size of chessboard

OUTPUT:

All valid N-Queen placements

BEGIN

CREATE board[N][N]

INITIALIZE board to EMPTY

CREATE columnUsed[N]

CREATE diagonal1Used[2*N - 1]

CREATE diagonal2Used[2*N - 1]

INITIALIZE all arrays to FALSE

solutionCount $\leftarrow$ 0

PROCEDURE BACKTRACK(row)

IF row = N THEN

 solutionCount $\leftarrow$ solutionCount + 1

 PRINT board

 RETURN

END IF

FOR col $\leftarrow$ 0 TO N - 1 DO

 diagonal1 $\leftarrow$ row - col + N - 1

 diagonal2 $\leftarrow$ row + col

 IF columnUsed[col] = FALSE AND

 diagonal1Used[diagonal1] = FALSE AND

 diagonal2Used[diagonal2] = FALSE THEN


```

// Place queen
board[row][col] ← 1

columnUsed[col] ← TRUE
diagonal1Used[diagonal1] ← TRUE
diagonal2Used[diagonal2] ← TRUE

// Move to next row
CALL BACKTRACK(row + 1)

// Backtrack
board[row][col] ← 0

columnUsed[col] ← FALSE
diagonal1Used[diagonal1] ← FALSE
diagonal2Used[diagonal2] ← FALSE

END IF

END FOR

END PROCEDURE

CALL BACKTRACK(0)

```

```
IF solutionCount = 0 THEN  
    PRINT "No valid solution exists"  
ELSE  
    PRINT "Total valid solutions = ", solutionCount  
END IF
```

END

Optimization used here

Instead of scanning the board every time, three arrays directly record whether a queen already occupies:

columnUsed

diagonal1Used

diagonal2Used

So an unsafe position can be **pruned immediately**.

19. Conclusion

The **N-Queens problem** can be effectively solved using the **Backtracking technique** by placing queens row by row and checking whether each position is safe. Whenever a placement violates the column or diagonal constraints, that branch of the search tree is immediately discarded. If a complete solution cannot be formed, the algorithm backtracks to the previous row and tries another position. Thus, backtracking provides a systematic and efficient method for finding **all valid N-Queen configurations** while avoiding unnecessary searches.